

□ Meta-Analisi per il Trauma Cranico: Spiegazione per Ragazzi del Liceo

Cosa Stiamo Studiando?

Immagina questa situazione: Sei al pronto soccorso. Entra un ragazzo che ha preso una botta alla testa giocando a calcio. Il medico deve decidere: “Gli faccio la TAC cerebrale o no?”

La TAC: - □ Costa molto - □ Espone a radiazioni (come tante radiografie insieme) - □ Spesso è negativa (non trova nulla)

Esistono dei test del sangue che potrebbero aiutare il medico a decidere! Questi test misurano delle proteine che escono dalle cellule cerebrali quando si fanno male.

I Due “Concorrenti”

Biomarcatore	Cosa misura	Età del test
GFAP + UCH-L1	Proteine del cervello danneggiato	□ Nuovo (2018)
S100B	Proteina delle cellule di supporto al cervello	□ Vecchio (1995)

□ Le Domande della Meta-Analisi

Una **meta-analisi** è come quando leggi 10 recensioni di un film prima di decidere se vederlo: raccogli le opinioni di tanti studi per avere una risposta più affidabile.

Le nostre domande sono: 1. **Quanto sono bravi questi test a trovare chi ha davvero una lesione?** 2. **Quanto sono bravi a escludere chi sta bene?** 3. **Quale dei due test è migliore?**

□ Concetti Base (Spiegati Semplicemente)

Il Test di Gravidanza come Esempio

Per capire, usiamo un esempio che conosci: il **test di gravidanza**.

	GRAVIDA	NON GRAVIDA
Test POSITIVO	□ TP	□ FP
Test NEGATIVO	□ FN	□ TN

- **TP** (True Positive): Test positivo, è davvero incinta □ □ Corretto!
- **FP** (False Positive): Test positivo, ma non è incinta □ □ Errore (falso allarme)
- **FN** (False Negative): Test negativo, ma è incinta □ □ Errore grave (non te ne accorgi!)
- **TN** (True Negative): Test negativo, non è incinta □ □ Corretto!

Sensibilità e Specificità

Sensibilità = “Quanto becco i malati?”

$$\text{Sensibilità} = \text{TP} / (\text{TP} + \text{FN})$$

Esempio: Se su 100 donne incinte, il test ne becca 95: - Sensibilità = 95% - 5 donne incinte ricevono un test negativo (pericoloso!)

Analogia: Un metal detector all'aeroporto. Se ha alta sensibilità, trova TUTTE le armi. Ma forse suona anche per le fibbie delle cinture...

Specificità = “Quanto escludo correttamente i sani?”

$$\text{Specificità} = \text{TN} / (\text{TN} + \text{FP})$$

Esempio: Se su 100 donne NON incinte, il test dice “negativo” a 90: - Specificità = 90% - 10 donne ricevono un falso positivo (preoccupazione inutile)

Analogia: Un filtro antispam. Se ha alta specificità, lascia passare SOLO le email importanti. Ma forse qualche spam passa...

Il Compromesso

C'è un **trade-off** (compromesso): se alzi la sensibilità, abbassi la specificità, e viceversa.

Esempio della vita reale: - Un portiere di calcio che esce sempre □ becca tutti i tiri (alta sensibilità), ma lascia la porta scoperta (bassa specificità) - Un portiere che resta sempre sulla linea □ difende bene la porta (alta specificità), ma i tiri lontani entrano (bassa sensibilità)

□ I Dati dei Nostri Studi

GFAP + UCH-L1 (8 studi, 4.550 pazienti)

Studio	TP	FP	FN	TN	N
Oris 2024	18	61	0	127	206
Bazarian 2021	24	199	1	167	391
Papa 2022	52	392	1	413	858
...	...	...	...	...	...

Cosa significano questi numeri? - In “Oris 2024”: 18 persone avevano la lesione e il test l'ha trovata (TP) - 61 persone erano sane ma il test ha suonato lo stesso (FP - falsi allarmi) - 0 persone avevano la lesione ma il test non l'ha vista (FN - pericoloso!) - 127 persone erano sane e il test ha detto sano (TN)

S100B (6 studi, 3.582 pazienti)

Studio	TP	FP	FN	TN	N
Oris 2024	7	148	0	51	206
Oris 2021	63	891	0	218	1.172
...	...	...	...	...	...

Nota: S100B ha molti più falsi positivi (FP alti) □ più allarmi inutili!

□ Il Codice Python Spiegato Passo dopo Passo

1. Importiamo gli “Attrezzi” (righe 13-24)

```
import numpy as np          # Per i calcoli matematici
import matplotlib.pyplot as plt # Per i grafici
from scipy.special import expit, logit # Funzioni magiche per le proporzioni
```

Spiegazione: È come quando apri la cassetta degli attrezzi. numpy è il martello per i numeri, matplotlib è il pennello per disegnare, scipy ha funzioni matematiche speciali.

2. I Colori per i Grafici (righe 44-50)

```
COL_GFAP = '#1B4F8A' # Blu scuro
COL_S100B = '#1A6B3C' # Verde scuro
```

Spiegazione: Scegliamo i colori come quando fai un poster per scuola. Blu per GFAP, verde per S100B. I colori sono in formato “esadecimale” (come #1B4F8A), un modo che i computer usano per dire “blu scuro”.

3. Inseriamo i Dati (righe 55-85)

```
GFAP_RAW = dict(
    study = ["Oris 2024", "Bazarian 2021", ...],
    TP = np.array([18, 24, 52, ...]),
    FP = np.array([61, 199, 392, ...]),
    FN = np.array([0, 1, 1, ...]),
    TN = np.array([127, 167, 413, ...])
)
```

Spiegazione: Creiamo una “tabella” con i dati. np.array crea una lista di numeri che il computer può manipolare facilmente.

4. La Funzione enrich() - Arricchiamo i Dati (righe 90-105)

```
def enrich(d, add=0.5):
    # Se una cella è 0, aggiungiamo 0.5
    tp[tp==0] += add
```

```
# Calcoliamo sensibilità e specificità
d['Se'] = tp/(tp+fn)
d['Sp'] = tn/(tn+fp)

# Trasformazione "logit" (spiegata sotto)
d['lSe'] = np.log(d['Se']/(1-d['Se']))
d['lSp'] = np.log(d['Sp']/(1-d['Sp']))
```

Perché aggiungere 0.5?

Immagina di avere 0 FN (False Negatives). Se fai $1/0$, il computer va in crash (divisione per zero!). Aggiungere 0.5 è come dire “tecnicamente quasi zero, ma calcolabile”.

Cos'è la trasformazione logit?

Le proporzioni (come $95\% = 0.95$) sono “scomode” per i calcoli statistici perché: - Non possono superare 100% o scendere sotto 0% - Sono “schiacciate” agli estremi

Il **logit** “srotola” questi numeri da $0-1$ a $-\infty$ a $+\infty$, rendendoli più facili da trattare.

```
logit(0.5) = 0          # Il centro
logit(0.9) = +2.2       # Vicino a 1
logit(0.1) = -2.2       # Vicino a 0
```

È come passare dalla scala Celsius alla scala Kelvin: entrambe misurano temperatura, ma Kelvin è più comoda per certi calcoli.

5. La Funzione DL() - Il Cuore della Meta-Analisi (righe 107-120)

```
def DL(y, v):
    k = len(y)                # Quanti studi abbiamo?
    w = 1/v                    # Peso = 1/varianza (più preciso = più peso)
    mFE = np.sum(w*y)/np.sum(w) # Media pesata (effetti fissi)
    Q = np.sum(w*(y-mFE)**2)    # Quanto gli studi "discrepano"
    t2 = max(0, (Q-(k-1))/C)    # Tau quadro: variabilità TRA studi
    wRE = 1/(v+t2)             # Nuovi pesi (effetti casuali)
    mu = np.sum(wRE*y)/np.sum(wRE) # Risultato finale!
```

Spiegazione semplice:

Questa funzione risponde alla domanda: “Dato che ogni studio dice qualcosa di leggermente diverso, qual è la ‘verità’ più probabile?”

Il concetto dei pesi: - Uno studio con 10.000 pazienti ha più “voce in capitolo” di uno con 100 pazienti - Uno studio molto preciso (varianza bassa) conta di più di uno impreciso

Effetti fissi vs effetti casuali:

Tipo	Pensiero	Esempio
Effetti fissi	“Tutti gli studi misurano ESATTAMENTE la stessa cosa”	Come misurare l’altezza della stessa persona con diversi strumenti
Effetti casuali	“Ogni studio misura qualcosa di SIMILE ma non identico”	Come misurare l’altezza di studenti di diverse scuole

Nella realtà, gli studi medici sono sempre un po’ diversi (pazienti diversi, ospedali diversi, metodi diversi), quindi usiamo **effetti casuali**.

I² (I-quadrato): “Quanta variabilità è VERA (non casuale)?” - I² = 0%: Gli studi sono tutti d’accordo (o quasi) - I² = 50%: Metà delle differenze sono reali, metà sono caso - I² = 75%: Gli studi sono molto diversi tra loro!

6. Gli Intervalli di Confidenza con Wilson (righe 122-125)

```
def wilson(p, n, z=1.96):
    # Calcola l'intervallo di confidenza di Wilson
```

Spiegazione: Quando uno studio dice “sensibilità 95%”, non è ESATTAMENTE 95%. È più tipo “tra il 90% e il 98%”. L’intervallo di confidenza ci dice: “Con il 95% di probabilità, il vero valore è tra X e Y”.

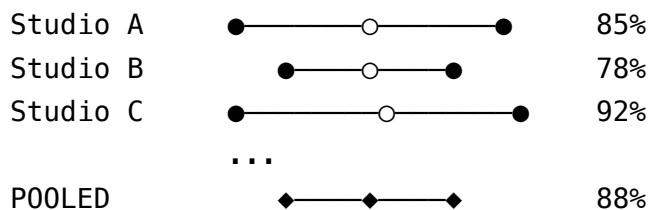
Il metodo di Wilson è più preciso del semplice “più o meno” quando i numeri sono vicini a 0% o 100%.

7. Creiamo i Grafici - Forest Plot (righe 251-378)

```
def forest_plot_pub(res, col_main, col_light, fname, fig_label):
    # Crea il grafico "forest plot"
    # Quadrati = studi singoli (dimensione = precisione)
    # Diamante = risultato combinato
```

Cos’è un Forest Plot?

È il modo standard di mostrare i risultati di una meta-analisi. Immagina una lista di studi, uno sotto l’altro:



- Il **punto** (●) è la stima dello studio
- La **linea** (—) è l’intervallo di confidenza
- Il **quadrato** (●) è più grande se lo studio è più preciso
- Il **diamante** (◈) in fondo è il risultato combinato di tutti gli studi

8. La Curva SROC (righe 380-455)

```
def sroc_points(d):  
    # Crea la curva SROC (Summary ROC)
```

Cos'è l'SROC?

SROC = Summary Receiver Operating Characteristic

Mostra il **compromesso** tra sensibilità e specificità: - In alto a sinistra = perfetto (100% Se, 100% Sp) - La diagonale = inutile come lanciare una moneta - Più la curva è vicina all'angolo in alto a sinistra, meglio è!

□ R vs Python: Qual è la Differenza?

Il Codice R (Versione "Pro")

```
fit_gfap <- reitsma(gfap, correction.control="all", correction=0.5)
```

R usa il **modello bivariato di Reitsma**, che è il "gold standard" (il migliore) per le meta-analisi diagnostiche.

Cosa fa di speciale? - Analizza Se e Sp **INSIEME** (non separatamente) - Stima la **correlazione** tra Se e Sp - Usa algoritmi sofisticati (massima verosimiglianza)

Il Codice Python (Versione "Semplificata")

```
rSe = DL(d['lSe'], d['vSe']) # Analisi sensibilità SEPARATA  
rSp = DL(d['lSp'], d['vSp']) # Analisi specificità SEPARATA
```

Python usa **DerSimonian-Laird marginale**: - Analizza Se e Sp **SEPARATAMENTE** - Ignora la correlazione tra Se e Sp - È un'approssimazione più semplice

Tabella Comparativa (Semplice)

Cosa	R (reitsma)	Python (DL)
Metodo	Bivariato completo	Due analisi separate
Correlazione Se-Sp	□ Sì, calcolata	□ No, ignorata
Precisione	□ □ □ □ □ Molto alta	□ □ □ □ Buona
Velocità	□ Più lenta	□ Istantanea
Facilità	□ Complessa	□ Semplice

Analogia per Capire

R (bivariato) è come un fotografo professionale che: - Usa una macchina fotografica costosa - Regola manualmente messa a fuoco, diaframma, tempi - Fa foto di qualità superiore - Richiede competenze tecniche

Python (DL marginale) è come uno smartphone che: - Fa foto buone automaticamente - È veloce e facile - Per la maggior parte degli usi va benissimo - Non è professionale, ma pratico

□ I Bias (Errori) di Usare Python

1. Bias di Approssimazione

Problema: Python analizza Se e Sp separatamente, ma nella realtà sono collegate!

Esempio: Se uno studio testa pazienti molto gravi: - Avrà alta sensibilità (trova tutte le lesioni gravi) - Ma bassa specificità (molti falsi positivi tra i meno gravi)

Ignorare questa connessione porta a intervalli di confidenza leggermente più ampi.

Quanto è grave? Per la maggior parte degli usi, poco. Le stime puntuali sono simili, solo gli intervalli sono un po' più conservativi.

2. Bias nella Curva SROC

Problema: Python usa il metodo MSL (Moses-Shapiro-Littenberg), che è un'approssimazione.

Esempio: È come disegnare una curva passando per 3 punti a occhio vs. usare un programma di geometria. Il risultato è simile, ma non identico.

3. Bias di “Fidarsi Troppo delle Figure”

Questo è il più pericoloso!

Python fa grafici bellissimi, ma: - I numeri potrebbero differire leggermente da R - Se guardi solo il grafico Python senza controllare i risultati R, rischi di sbagliare

Soluzione: 1. Fai l'analisi principale con R 2. Usa Python SOLO per le figure 3. Controlla che i numeri siano simili!

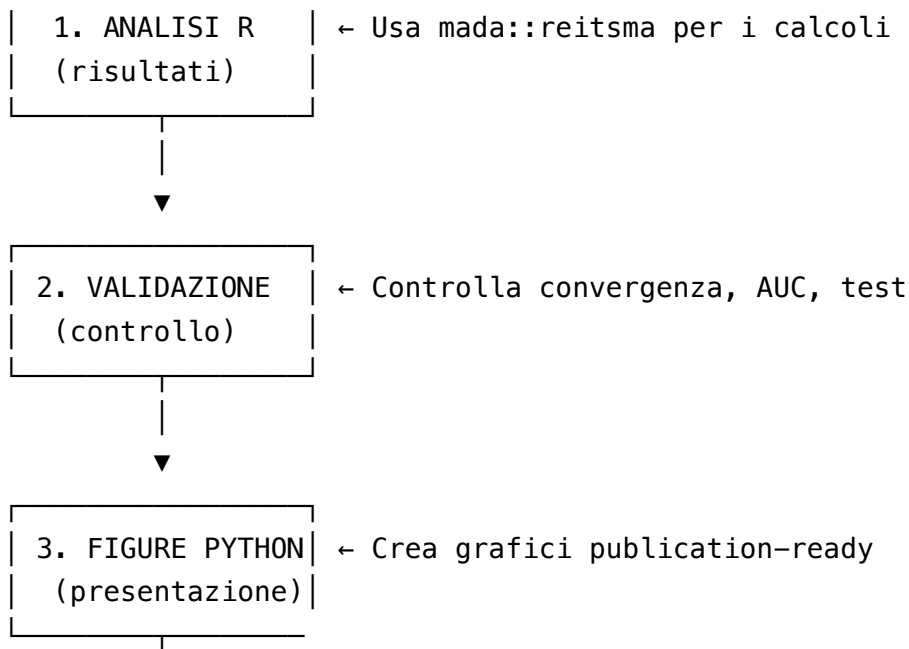
4. Cosa Manca in Python?

Cosa	R	Python
AUC (Area Under Curve)	□ Sì	□ No
Test di bontà del fit	□ Sì	□ No
Analisi dei residui	□ Sì	□ No
Controllo convergenza	□ Automatico	□ Manuale

AUC = Area Under the Curve. Dice quanto è “buono” il test in generale (0.5 = inutile, 1.0 = perfetto).

□ Workflow Consigliato (Come Lavorare Bene)

Il Metodo “Best Practice”



Checklist Prima di Pubblicare

- ☐ Le stime puntuali (Se, Sp) differiscono meno del 2% tra R e Python?
- ☐ Gli intervalli di confidenza sono simili?
- ☐ L' I^2 (eterogeneità) è coerente?
- ☐ La curva SROC passa vicino al punto pooled?
- ☐ Hai citato entrambi i metodi nel paper?

□ Glossario (Parole Nuove Spiegate)

Parola	Significato Semplice
Meta-analisi	Unire i risultati di tanti studi per una risposta più sicura
Sensibilità	Capacità di trovare chi ha il problema
Specificità	Capacità di escludere chi NON ha il problema
TP/FP/FN/TN	True/False Positives/Negatives (veri/falsi positivi/negativi)
Logit	Trasformazione matematica che "srotola" le percentuali
I^2	Quanto gli studi "veramente" divergono (0-100%)
τ^2 (tau quadro)	Quanto gli studi "veri" differiscono tra loro
Effetti fissi	Tutti gli studi misurano la stessa cosa esatta
Effetti casuali	Ogni studio misura qualcosa di leggermente diverso
Forest plot	Grafico con tanti punti che mostra i risultati
SROC	Curva che mostra il compromesso Se vs Sp
NPV	Se il test è negativo, quanto è probabile che io stia bene?
Correzione di continuità	Aggiungere 0.5 per evitare divisioni per zero
Bias	Errore sistematico che distorce i risultati
AUC	Punteggio di quanto è buono un test (0.5-1.0)

□ Conclusione: Cosa Abbiamo Scoperto?

Risultati Principali

Biomarcatore	Sensibilità	Specificità	NPV
GFAP+UCH-L1	~98%	~60%	>99%
S100B	~95%	~30%	>99%

Cosa Significa in Pratica?

GFAP+UCH-L1 è migliore perché: - Ha meno falsi positivi (60% vs 30% di specificità) - Quindi evita più TAC inutili - Ma entrambi sono eccellenti per escludere lesioni (NPV >99%)

Il Messaggio Clinico

Questi test sono **perfetti per “escludere”** (regola-out): - Se il test è negativo, puoi stare tranquillo (>99% probabilità di non avere lesioni) - Se il test è positivo, serve la TAC per confermare

Non sono buoni per “diagnosticare” (regola-in) da soli.

Il Messaggio Metodologico

Aspetto	Raccomandazione
Analisi statistica	Usa R con <code>mada::reitsma</code>
Figure	Usa Python per grafici publication-ready
Validazione	Confronta sempre i risultati
Pubblicazione	Cita entrambi i metodi

Python è fantastico per le figure, ma R rimane il re per l'analisi statistica delle meta-analisi diagnostiche!

□ Domande che Potresti Averti

“Perché non usare solo Python se è più semplice?”

Perché la scienza richiede precisione. Python è un'approssimazione buona, ma R è il “gold standard”. Nella ricerca medica, la precisione può salvare vite!

“Posso fidarmi dei risultati Python per un compito di scuola?”

Sì! Per scopi didattici e comprensione, Python è perfetto. Le differenze sono piccole e non cambiano le conclusioni generali.

“Devo imparare R o Python?”

Entrambi! - Python è più versatile (intelligenza artificiale, web, automazione) - R è specializzato per la statistica - Conoscere entrambi ti dà superpoteri! □

Documento creato per scopi didattici. Per la ricerca scientifica vera, consulta sempre un biostatistico!